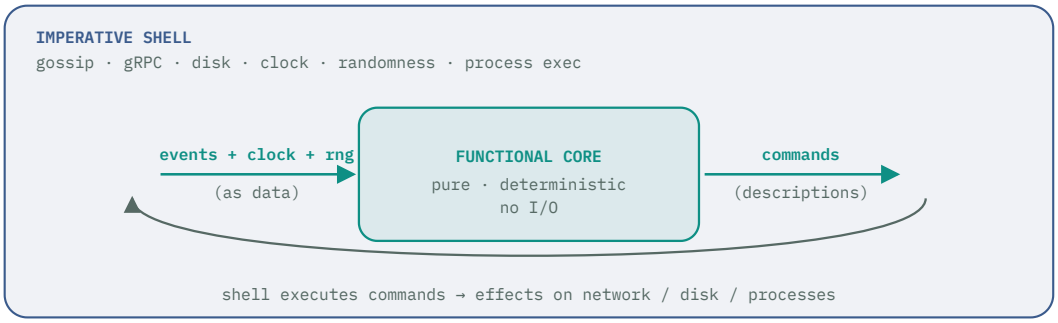
COMPONENT DESIGN · PHASE 0

The P0 build, split into pure cores and thin shells.

Every P0 component is designed as a **functional core** that decides and an **imperative shell** that acts. The core is pure and exhaustively testable; the shell is thin and does all the I/O. This doc gives each component its responsibility, its core functions, its shell, and the data crossing between.

01 The FCIS pattern, applied to Swarm

The dividing line for Swarm is **decisions vs. actions**. The **functional core** takes plain data and returns *descriptions of what to do* — commands — and touches nothing else. The **imperative shell** runs a loop: gather events, call the core, execute the commands it returns. The one rule that keeps the core pure: it never *performs* an effect and never reads the clock or a random source itself — the shell reads those and passes them **in as data**, so the core stays deterministic and every decision is reproducible in a unit test.



Data in, commands out; the shell does the rest. The core decides and returns commands; the shell is the only thing that touches the outside world. Because time and randomness enter as arguments, a decision can be replayed exactly in a test with no network, disk, or clock.

Why it's worth it here. The decisions in Swarm — when a cell splits, where a task lands, whether a quorum agrees, how a barrier steps — are the parts most likely to be subtly wrong and hardest to reproduce through live I/O. As pure functions they get exhaustive unit tests; the thin shell needs only a handful of integration tests.

02 How to read each component

Every component below is specified the same way, so the core/shell boundary is explicit and implementable:

FIELD	WHAT IT GIVES YOU
Responsibility	the one job this component owns
Functional core	the pure functions, with input/output types — the decisions, in signature form
Imperative shell	the I/O it performs and the run loop that feeds the core and executes its commands
Boundary types	the plain data that crosses between core and shell

Signatures are written in a Go-flavored pseudocode; sum types (e.g. `Split | Merge | None`) are shown as tagged unions. Go packages hold the shells and the pure cores side by side (a `core` package that imports no `net` / `os`); Rust carries the hot-path cores in later phases.

03 Shared data model

The types that cross component and core/shell boundaries. They are plain data — no methods that do I/O — so the core can take and return them freely.

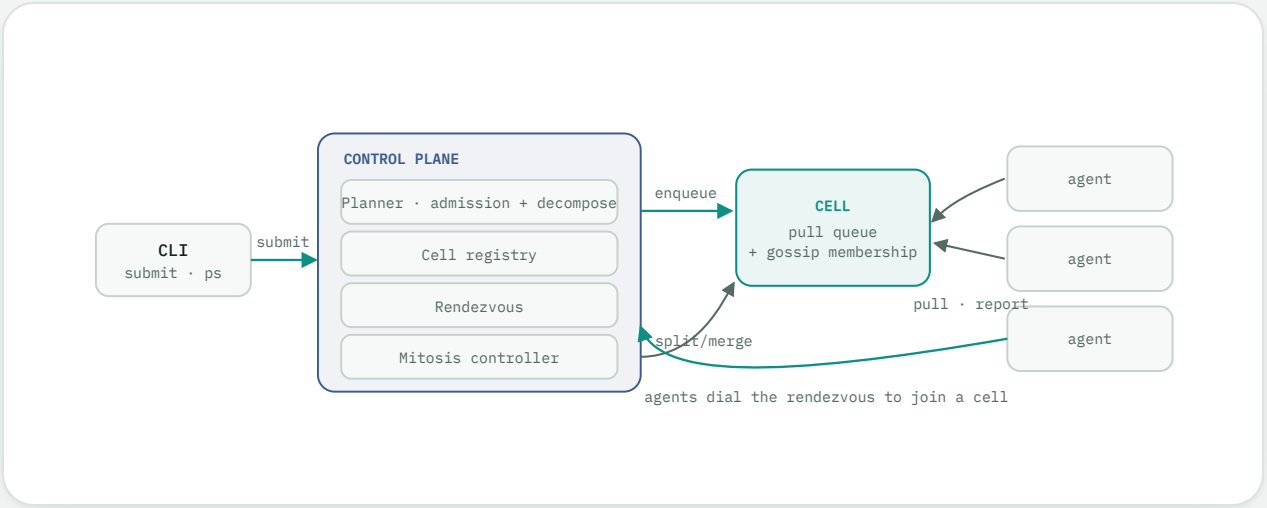
core/model.go — boundary types

```
type JobSpec    struct { ID JobID; Template string; Coupling Coupling; Params r
type Task       struct { ID TaskID; JobID JobID; Input []byte; Attempt int }
type TaskResult struct { TaskID TaskID; Output []byte; OK bool }
type Aggregate  struct { JobID JobID; Value []byte; Done bool }

// what the planner/mitosis reason over — snapshots, not live handles
type CellView   struct { ID CellID; Size int; Free int; Caps CapSet }
type Instant    int64   // monotonic ns, supplied by the shell

// P0 coupling is always Independent; the field exists for later phases
type Coupling   int     // Independent | Barrier | Leader | MessagePassing
```

Commands are data too. Each core returns a slice of command values (e.g. `Dispatch`, `SplitCell`, `Dial`) that the shell pattern-matches and executes. A core that returns commands instead of calling them is what makes it pure — and what lets a test assert on the commands without anything actually happening.

04 P0 system map

P0 is one region, trusted machines, native execution, the `independent` driver only. Five moving parts: the CLI, the control plane (planner, registry, rendezvous, mitosis), and the agent — over a cell that is just a pull queue plus gossip membership.



Submit flows left-to-right; agents dial in from the right. The control plane plans and places; the cell holds the queue; agents self-register, pull, execute, and report. The mitosis controller only watches cell sizes and issues split/merge.

05 Components

CLI CLIENT

Responsibility — turn a user command into a control-plane request and render the reply.

◆ FUNCTIONAL CORE · PURE

```
func parse(argv []string) (Command, error)
func validate(spec JobSpec) error
func render(reply Reply) string
```

▟ IMPERATIVE SHELL · I/O

- read `argv`, env, and any job file
- open the gRPC channel, send the request
- stream/print the rendered output to the terminal

Boundary — `Command`, `JobSpec` in; `Reply` out. The core never opens a socket; it only decides what to send and how to show the answer.

Planner & admission CONTROL PLANE

Responsibility — accept or reject a job, decompose it into tasks, and decide which cell each task goes to.

◆ FUNCTIONAL CORE · PURE

```
func admit(spec JobSpec) ([]Task, Reject)
  // validates, then calls the template's decompose()
func place(t Task, cells []CellView) Placement
  // Placement = Assign{CellID} | NoCapacity
```

▟ IMPERATIVE SHELL · I/O

- receive the `Submit` RPC
- persist job + tasks to the store
- enqueue each placed task on its cell
- on `NoCapacity`, hold and retry on the next registry tick

Boundary — `JobSpec` + `[]CellView` snapshot in; `[]Task` + `Placement` decisions out. Placement in P0 is "any cell with free capacity," so it's a pure function of the snapshot.

Cell registry CONTROL PLANE

Responsibility — hold the authoritative set of cells and their capacities, updated from membership events.

◆ FUNCTIONAL CORE · PURE

```
func apply(reg Registry, ev RegistryEvent) (Registry, []Change)
  // ev = CellUp | CellDown | CapacityChanged | AgentJoined | AgentLeft
func snapshot(reg Registry) []CellView
```

▟ IMPERATIVE SHELL · I/O

- subscribe to gossip / rendezvous events
- call `apply`, persist the new `Registry`
- publish `snapshot()` for the planner and mitosis to read

Boundary — `RegistryEvent` in; next `Registry` + `[]Change` out. The registry value is immutable data; the shell swaps the stored pointer.

Rendezvous CONTROL PLANE

Responsibility — accept agents that dial in, admit them, and assign each to a cell.

◆ FUNCTIONAL CORE · PURE

```
func admitAgent(req JoinReq, reg []CellView) AdmitDecision
  // AdmitDecision = Accept{CellID} | Reject{reason} | NewCell
```

▟ IMPERATIVE SHELL · I/O

- terminate the agent's mTLS dial-out (gRPC stream)
- call `admitAgent` with a registry snapshot
- on `Accept`, wire the agent into the cell's gossip
- emit an `AgentJoined` event to the registry

Boundary — `JoinReq` + snapshot in; `AdmitDecision` out. Whether to open a new cell (first agent) or slot into an existing one is a pure call.

Mitosis controller CONTROL PLANE

Responsibility — decide when a cell splits or two merge. In P0 (independent jobs) resize is safe any time, so this is the whole differentiator in its simplest form.

◆ FUNCTIONAL CORE · PURE

```
func decide(cells []CellView, cfg Thresholds,
            cooldowns map[CellID]Instant,
            now Instant) []Mitosis
  // Mitosis = Split{CellID} | Merge{a,b CellID} | None
  // split when Size > 2T; merge two neighbors < T;
  // suppress if now-cooldown[id] < window
```

▟ IMPERATIVE SHELL · I/O

- read the registry snapshot + clock on a timer
- call `decide`
- execute `Split` / `Merge`: reconfigure cells, reassign agents, stamp cooldowns

Boundary — sizes, thresholds, cooldowns, and `now` in; a list of `Mitosis` commands out. Every split/merge rule (the 2T band, cooldown, sustained window) is unit-tested with hand-built snapshots and a fake clock — no cluster required.

Agent · registration & reconnect AGENT

Responsibility — get and stay joined: dial out, enroll once, join a cell, heartbeat, and resume the loop on any drop.

◆ FUNCTIONAL CORE · PURE

```
func step(s RegState, ev RegEvent,
          now Instant, jitter float64) (RegState, []RegCmd)
  // RegState = Dialing | Enrolling | Joining | Member
  // RegEvent = DialOK | DialFail | Enrolled | Joined
  //          | ConnLost | Tick
  // RegCmd   = Dial{region} | Enroll | JoinCell
  //          | Heartbeat | Wait{d} | Failover
func backoff(attempt int, cfg BackoffCfg,
             jitter float64) Duration
```

▟ IMPERATIVE SHELL · I/O

- perform `Dial` / `Enroll` / `Heartbeat` over mTLS gRPC
- read the clock; draw the `jitter` and pass it in
- run `Wait` timers; detect dropped connections → `ConnLost`
- hold the SPIFFE identity across reconnects

Boundary — `(state, event, now, jitter)` in; `(next state, commands)` out. The retry-forever + failover behavior is a pure state machine, so "connection lost while a Member → dial again, keep identity" is a two-line test.

Agent · task runner AGENT

Responsibility — pull a task, run it natively, report the result; re-pull when idle.

◆ FUNCTIONAL CORE · PURE

```
func step(s RunState, ev RunEvent) (RunState, []RunCmd)
  // RunEvent = Idle | Pulled{task} | Done{result} | Failed
  // RunCmd   = Pull | Execute{task} | Report{result}
  //          | ReQueue{task}
```

▟ IMPERATIVE SHELL · I/O

- `Pull` from the cell queue over gRPC
- `Execute`: spawn the native process, capture output
- `Report` the `TaskResult` back

Boundary — `RunEvent` in; `RunCmd` out. Membership itself lives in the shell (the `memberlist`/SWIM library); the pure part is `reconcile(view, events) → (view, changes)`, how gossip events become a cell view.

Templates · keyspace-search, monte-carlo PURE LOGIC

Responsibility — split a job into independent tasks and merge their results. Templates are the purest illustration of the pattern: *all core, no shell.*

◆ FUNCTIONAL CORE · PURE

```
// keyspace-search
func decompose(j KeyspaceJob) []Task   // split range
func merge(rs []TaskResult) Aggregate  // first-hit

// monte-carlo
func decompose(j MCJob) []Task         // seed blocks
func merge(rs []TaskResult) Aggregate  // sum · stats
```

▟ IMPERATIVE SHELL · I/O

- *none.* Templates are pure functions the planner (`decompose`) and the job tracker (`merge`) call.
- The workload binary they describe is what the agent's shell executes.

Boundary — a typed job in; `[]Task` or an `Aggregate` out. A new template is a new pair of pure functions registered by name — no shell to touch.

06 What this buys, and what P0 defers

The payoff is a testing split that matches the risk. The **cores** — mitosis, placement, admission, the two agent state machines, the templates — get exhaustive table-driven unit tests with hand-built inputs and a fake clock, because they're pure. The **shells** are thin enough to cover with a handful of integration tests: a real dial-out, a real pull, a real split executed against a live registry.

> **Deferred by design (later phases keep the same boundary).** Checkpointing and the barrier/leader/message-passing drivers (P2), multi-region and the regional control plane (P1), WASM sandboxing and quorum/reputation verification (P3) all slot in as new cores (decisions) and new shell actions — they don't move the FCIS line, they extend it. The `Coupling` field is already in the model so P2 has nothing to retrofit.
